

GCN-Based Bundle Pricing: Strategies and Experiments

Chenghan Wu, Liangyu Ding

1 Background

1.1 Applications of Bundle Pricing

Bundle pricing is a widely adopted pricing strategy across various industries. Its core idea is to sell multiple products or services as a single package at a unified price to maximize revenue or achieve market penetration. This approach is commonly used in software, communication services, travel packages, digital media subscriptions, insurance, and retail.

1.2 Mainstream Methods for Solving Bundle Pricing

Currently, both academia and industry primarily rely on traditional optimization methods to solve bundle pricing problem:

- **Hanson and Martin (1990): MILP-Based Optimal Bundle Pricing**
Hanson and Martin were the first to systematically formulate the Bundle Pricing problem as a *Mixed Integer Linear Programming (MILP)* model. Their approach explicitly encodes customer self-selection constraints, product costs, and segment-specific reservation prices, aiming to maximize the seller's profit. While theoretically rigorous and highly general, the model suffers from the curse of dimensionality: as the number of products n increases, the feasible bundle space grows exponentially (2^n), making it **computationally intractable for large-scale instances**.
- **Li et al. (2021): Convex Optimization for Bundle Size Pricing**
To address MILP scalability issues, Li et al. proposed a simplified version of the problem by modeling the bundle price as a *convex function of bundle size*, assuming that customer utility and purchase decisions are size-dependent rather than composition-specific. This reformulation enables the use of efficient convex optimization methods and significantly reduces the problem dimension. However, it **ignores product heterogeneity** and assumes homogeneous valuation over item combinations of the same size, thereby limiting its precision and applicability in personalized or item-differentiated markets.

1.3 GCNs in Revenue Management

Graph Neural Networks (GCNs) have emerged as powerful tools for modeling relational data in operations research and revenue management. Unlike traditional models, GCNs

naturally operate on *graph-structured data*, where entities such as products and customer segments are represented as nodes, and their interactions (e.g., similarity, co-purchase behavior) are encoded as edges.

A typical GCN layer aggregates information from a node’s local neighborhood using neural message-passing schemes. This enables the model to capture both local and global structural patterns. The most common architectures include Graph Convolutional Networks (GCNs) and GENConv, each differing in how they weigh and combine neighbor information.

The training of GCNs typically involves supervised learning, where node-level or edge-level labels (e.g., purchase likelihood, utility score) are predicted. Input features may include customer profiles, product attributes, and historical interaction data, while outputs are typically probability scores or embeddings used in downstream optimization tasks.

Notable works such as Fan et al. and Liu & Ye demonstrate how GCNs accelerate LP solvers by learning optimal pivots or basis selection.

2 Proposed GCN-Based Strategies

Using GENConv as the graph backbone, we design four strategies to solve the Bundle Pricing problem. The model predicts a customer-product probability matrix $P \in \mathbb{R}^{m \times n}$ where P_{ij} is the probability that segment j selects product i . The probability matrix will be used for bundle space pruning based on different strategies.

Data Generation

First, we sample the ship costs $c_k^{\text{ship}} \sim \mathcal{U}[0, 1]$. Based on our assumption that the customer types are fixed, the ship cost for each product remains static during sample iterations. In each sample iteration, we first draw the number of products $n \sim \mathcal{U}\{l, \dots, u - 1\}$. Next, we sample the unit costs $c_j^{\text{unit}} \sim \mathcal{U}[0, 1]$ and form the assortment matrix $A = (a_{b,i}) \in \{0, 1\}^{B \times n}$, where the b -th row is the binary expansion of $b - 1$. The total cost vector is then $\mathbf{c} = 0.1(A\mathbf{c}^{\text{unit}} + \mathbf{c}^{\text{ship}})$.

We sample unit utilities $U_{kj} \sim \mathcal{U}[0, 1]$, and compute the bundle-utility $R_{kb} = f(U_{k,:} A_{:,b})$, where $f(\cdot)$ is a concave function.

Finally, we draw the customer proportion $\boldsymbol{\alpha} = (\alpha_1, \dots, \alpha_m)$, where $\alpha_i = \frac{\alpha'_i}{\sum_{l=1}^m \alpha'_l}$, $\alpha'_i \sim \mathcal{U}[0, 1]$.

$c_k^{\text{ship}} \sim \mathcal{U}[0, 1]$	Shipping cost for customer type k , fixed during all iterations.	(1)
$n \sim \mathcal{U}\{l, \dots, u - 1\}$	Number of products sampled in each iteration.	(2)
$c_j^{\text{unit}} \sim \mathcal{U}[0, 1]$	Unit cost for product j .	(3)
$A = (a_{b,i}) \in \{0, 1\}^{B \times n}$	Assortment matrix, b -th row is binary expansion of $b - 1$.	(4)
$\mathbf{c} = 0.1(A \mathbf{c}^{\text{unit}} + \mathbf{c}^{\text{ship}})$	Total cost vector for each bundle.	(5)
$U_{kj} \sim \mathcal{U}[0, 1]$	Unit utility of product j for customer type k .	(6)
$R_{kb} = f(U_{k,:}, A_{:,b})$	Bundle utility for customer k , where $f(\cdot)$ is concave.	(7)
$\boldsymbol{\alpha} = (\alpha_1, \dots, \alpha_m)$	Customer proportion vector.	(8)
$\alpha_i = \frac{\alpha'_i}{\sum_{l=1}^m \alpha'_l}, \quad \alpha'_i \sim \mathcal{U}[0, 1]$	Normalized random customer share.	(9)

Graph Representation

The figure below is the graph representation of optimal bundle pricing problem, which serves as the input of the GCN. Two types of nodes are included in the graph representation: customer nodes ($\bigcirc$), and product nodes ($\square$), as shown in Figure 1.

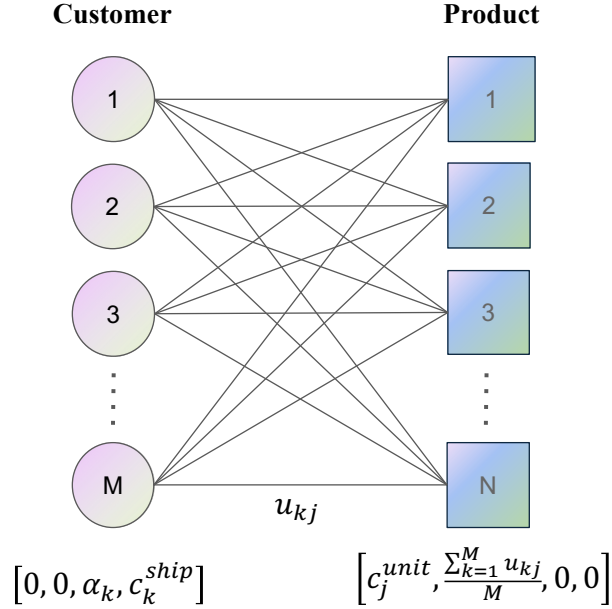


Figure 1: Graph representation of optimal bundle pricing problem.

In Figure 1, let $\mathcal{V}, \mathcal{E}$ denote the sets of nodes and edges, respectively. Let $\mathbf{Y}^S$ and $\mathbf{Y}^P$ denote the feature matrices of customer nodes and product nodes respectively. We represent each node with a vector of size $d^V = 4$, which captures utilities, costs, and market composition: the feature vector of each customer k is $\mathbf{Y}_{k,:}^S = [0, 0, \alpha_k, c_k^{\text{ship}}]$; the

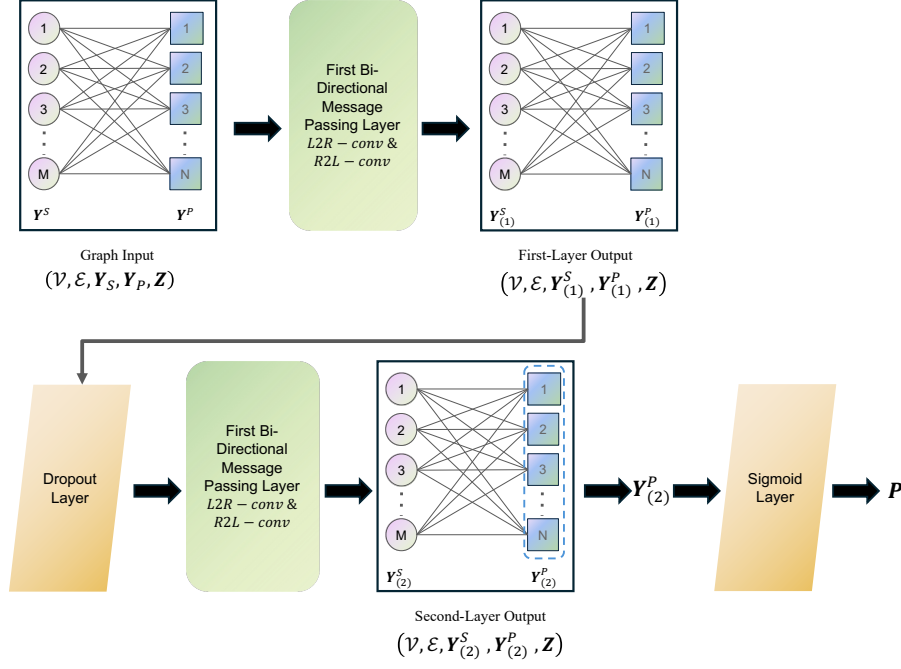


Figure 2: Illustration of GCN network.

feature vector of each product j is $\mathbf{Y}_{j,:} = [c_j^{\text{unit}}, \frac{1}{M} \sum_{k=1}^M u_{kj}, 0, 0]$. Let $\mathbf{Z}$ denote the feature matrix of edges. Each edge is characterized by a scalar u_{kj} , representing the k -th customer's utility towards product j .

GCN

We construct a GCN including two Bi-Directional message-passing layers, one dropout layer, and one Sigmoid layer, as shown in 2. We explain the computation details below.

First Bi-directional message-passing layer. The first message-passing layer in Figure 2 maps the graph in Figure 1 into a new graph with updated node features and the same edge features.

Denote the set of all products by P and the set of all customers by S . We then define the forward edge set $\vec{\mathcal{E}} = \{(i, j) \mid i \in P, j \in S\}$, and the backward edge set $\overleftarrow{\mathcal{E}} = \{(j, i) \mid (i, j) \in \vec{\mathcal{E}}\}$.

We arrange the product nodes first and the segment nodes afterwards; stack the corresponding row vectors to obtain the feature matrix

$$X = \begin{bmatrix} \mathbf{Y}_{1,:}^P \\ \vdots \\ \mathbf{Y}_{n,:}^P \\ \mathbf{Y}_{1,:}^S \\ \vdots \\ \mathbf{Y}_{m,:}^S \end{bmatrix} \in \mathbb{R}^{(n+m) \times 4}.$$

Thus $X_{j,:} = \mathbf{Y}_{j,:}^P$ for $j \leq n$ (product rows) and $X_{n+k,:} = \mathbf{Y}_{k,:}^S$ for $k = 1, \dots, m$ (segment rows).

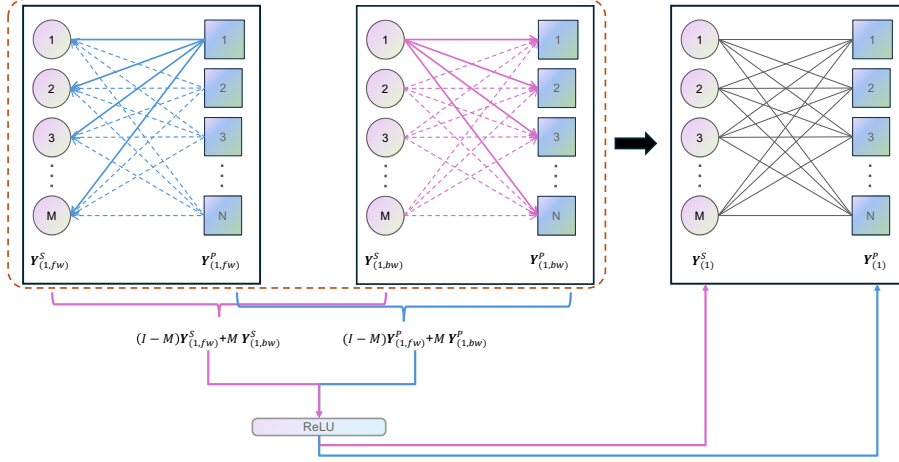


Figure 3: Bi-Directional Message Passing

The forward-pass node features are computed as $X_{\text{fw}}^{(1)} = \text{GENConv}(X, \vec{\mathcal{E}}, \mathbf{Z})$, and the backward-pass node features as $X_{\text{bw}}^{(1)} = \text{GENConv}(X, \overleftarrow{\mathcal{E}}, \mathbf{Z})$.

Let $m = (m_1, \dots, m_N)^\top$ be the mask vector with $m_i = \mathbb{1}\{i \in P\}$. We merge the two directions at each node i by

$$\tilde{X}_i^{(1)} = (1 - m_i) X_{\text{fw},i}^{(1)} + m_i X_{\text{bw},i}^{(1)},$$

and apply a ReLU function $\text{ReLU}(\mathbf{x}) = \max\{\mathbf{x}, 0\}$: $X_i^{(1)} = \text{ReLU}(\tilde{X}_i^{(1)})$,

Let $M = \text{diag}(m_1, \dots, m_N) \in \mathbb{R}^{N \times N}$ the associated diagonal matrix. With $X_{\text{fw}}^{(1)}, X_{\text{bw}}^{(1)} \in \mathbb{R}^{N \times d}$, we write in matrix form alternatively, as shown in 3,

$$X^{(1)} = \text{ReLU}((I - M) X_{\text{fw}}^{(1)} + M X_{\text{bw}}^{(1)}).$$

where $\text{ReLU}(Z)$ acts entry-wise as $\max\{z, 0\}$.

and finally use dropout with rate $p = 0.5$: $\hat{X}^{(1)} = \text{Dropout}(X^{(1)}, p = 0.5)$. This $\hat{X}^{(1)}$ then serves as the input to the next layer.

Second Bi-directional message-passing layer. The construction of second Bi-directional message-passing layer is similar to the first message-passing layer. The only difference is that the input sizes and the output size are (N, d_{mid}) , and $(1, M)$ respectively. Thus, the size of output matrix $\mathbf{Y}_{(2)}^P$ is $N \times M$.

3 Strategy Definition

Traditional methods for solving the Bundle Pricing problem, such as the MILP formulation proposed by Hanson, face two main challenges: (1) the combinatorial explosion of the feasible bundle space, which grows exponentially with the number of products (2^n); and (2) the computational burden of solving mixed-integer programs, where the presence of integer variables significantly slows down the optimization process relative to continuous LPs.

To overcome these challenges, we propose a learning-augmented optimization framework that uses Graph Neural Networks (GCNs) to predict segment-product preference probabilities. The key motivation is to use these predictions to construct a compact, high-quality approximation of the optimal bundle space, thereby improving tractability without heavily compromising solution quality.

- **Threshold Pruning Method(TPM)** constructs one bundle per segment by applying a fixed threshold (e.g., 0.5) to the predicted probability matrix. These m bundles form a pruned candidate set that is shared globally across all segments in the MILP. This approach effectively reduces the feasible space from 2^n to m , and removes one-to-one assignment between segment and bundle.
- **TopM Pruning Method(MPM)** generalizes the pruning scheme by expanding each segment’s preference ranking into m prefix bundles (e.g., top-1, top-2, ..., top- m), resulting in an $m \times m$ bundle space. This offers more flexibility at the cost of moderate computational overhead, while still keeping the feasible space independent of the full 2^n scale.
- **Percentage Pruning Method(PPM)** delete all products that lies in the last (1-k) quantile of all products, and include all subsets of the remaining products. The strategy is intended to include more possible cases for relatively good products. However, for small cases, solving MILP directly is the best choice, and for large cases, the additional possibilities, with high computational costs, is not worthwhile. The order of probability is more important.

In addition to pruning, we further explore the possibility of *fixing* segment-bundle assignments directly based on the GCN predictions.

- **Linear Program with Local Search(LPLS)** Bypasses the MILP formulation during initial optimization and solves a relaxed Linear Program (LP) with fixed binary bundle assignments. Since integer decision variables are removed, this approach offers significantly faster solving times. However, prematurely fixing segment-bundle pairs may degrade revenue or face infeasible scenarios. To mitigate this, we incorporate a greedy local search: starting from the thresholded bundles, it iteratively adds or removes products based on LP-evaluated revenue gains until no improvement is found.

The details of strategies’ implementation is attatched in appendix

Bundle Pricing Strategy Comparison (Example with $m = 3, n = 4$)

GEN-predicted probability matrix:

$$\mathbf{P} = \begin{bmatrix} 0.8 & 0.6 & 0.2 & 0.4 \\ 0.4 & 0.6 & 0.2 & 0.1 \\ 0.6 & 0.3 & 0.9 & 0.2 \end{bmatrix}$$

Method Comparison Table

Method	Bundle Space Definition	Size	Solving
Hanson	All $2^4 - 1 = 15$ bundles	15	1 MILP
BSP	All $2^4 - 1 = 15$ bundles with same-size pricing	15	1 MILP
TPM	Segment-wise threshold: $P_{ij} \geq 0.5$	3	1 MILP (M)
MPM	Segment-wise Top- m products, incremental bundles	9	1 MILP (M*M)
PPM	Power Set of the first 60% products	11	1 MILP ($\leq m \times 2^k$)
LPLS	Threshold-based + local search (add/remove product)	variable	multiple LP

TPM: Threshold-based Pruning

MPM: TopM-based Pruning

PPM: Percentage-based Pruning

LPLS: Linear Program with local search

When $m \geq 5$, MPM's problem size will be less than Hanson and BSP ($m * m \leq 2^m$)

Strategy Construction Examples (Based on P)

- **TPM:** Predicted bundles with probability ≥ 0.5 :

Segment 0: [1, 1, 0, 0]

Segment 1: [0, 1, 0, 0]

Segment 2: [1, 0, 1, 0]

- **MPM:** Top-3 products per segment, generate incremental bundles:

Segment 0: [0] $\rightarrow$ [0, 1] $\rightarrow$ [0, 1, 3]

Segment 1: [1] $\rightarrow$ [1, 0] $\rightarrow$ [1, 0, 2]

Segment 2: [2] $\rightarrow$ [2, 0] $\rightarrow$ [2, 0, 1]

Although Strategy A and Strategy B have one-to-one correspondence bundles, we solve the MILP problem by allowing each segment to choose other bundles freely.

- **PPM:** Top 3 products per segment, generate power set:

Segment 0: [0], [1], [3], [0, 1], [0, 3], [1, 3], [0, 1, 3]

Segment 1: [0], [1], [2], [0, 1], [0, 2], [1, 2], [0, 1, 2]

Segment 2: [0], [1], [2], [0, 1], [0, 2], [1, 2], [0, 1, 2]

Here the bundle set will be [0],[1],[2],[3],[0,1],[0,2],[0,3],[1,2],[1,3],[0,1,2],[0,1,3]

- **LPLS:** Initialize with Strategy A, perform local search on each segment by:

– On segment0:

Adding the highest-probability unselected product:[1,1,0,1]

Removing the lowest-probability selected product:[1,0,0,0]

– On segment1(when the best for segment0 is found):

...

Evaluate each neighbor with LP, select the best local optimum.

4 Experimental Results

4.1 Experiment Design

We evaluate our proposed strategies on a set of datasets generated using the data generation method described in Section 2. Specifically, we construct small-scale product instances where the number of products $n \in [5, 10]$. For each instance, we solve the original MILP formulation proposed by Hanson to obtain the optimal bundle assignments (`opt_bundle`), optimal prices (`opt_price`), optimal revenue (`opt_revenue`), and solving time (`running_time`).

The optimal bundles serve as labels for supervised training of the GCN-based prediction model. During evaluation, we use `opt_revenue` as the benchmark for assessing the effectiveness of each strategy’s revenue outcome, and `running_time` as the benchmark for computational efficiency. We define two normalized performance metrics:

- **Revenue Ratio:** $\text{revenue_ratio} = \frac{\text{strategy_revenue}}{\text{opt_revenue}}$
- **Time Ratio:** $\text{time_ratio} = \frac{\text{strategy_time}}{\text{running_time}}$

These metrics allow us to directly compare the solution quality and speed of our GCN-based strategies to the exact MILP solver under identical instance settings.

However, we observe that for larger problem sizes ($n > 13$), the MILP solver often fails to return feasible solutions within acceptable time limits or encounters infeasibility due to exponential growth in the bundle space. To accommodate this, for the case of $n = 14$, we adopt the Bundle Size Pricing (BSP) method proposed by Xiaobo Li as a proxy benchmark to compute `opt_revenue` and `running_time` in large-scale settings.

Note that the GCN model is still trained using data labeled by MILP-based optimal solutions on small instances. The trained model is then transferred and tested on large-scale samples to evaluate the generalization of each strategy.

The details of the definition of the MILP method and the BSP method are attached in the appendix

4.1.1 Prediction Loss

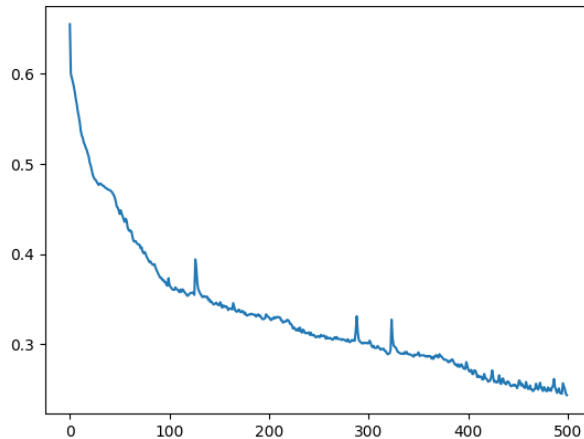


Figure 4: Train-loss

4.2 Small- and Large-Scale Experiments

Table 1: Performance of Strategies under small-scale ($5 < n < 10$) and large-scale ($n = 14$) settings.

Strategy	5 < n < 10 (Base: Full MILP, Base time = 0.9600s)			n = 14 (Base: BSP, Base time = 604.7870s)		
	Revenue Ratio	Time Ratio	Time (s)	Revenue Ratio	Time Ratio	Time (s)
TPM	0.8876	0.0947	0.0352	1.0613	0.0002	0.1441
MPM	0.9795	0.5905	0.2465	1.1408	0.0009	0.5238
PPM ($\lambda\% = 60\%$)	0.9513	0.5292	0.3691	1.1581	0.0262	15.8742
LPLS(e^{-2})	1.0317	0.2085	0.2002	1.1519	0.0064	3.8706
LPLS(e^{-4})	1.0484	0.4928	0.4731	1.1713	0.0392	23.7077

Here LPLS method meets the problem of revenue ration ≥ 1 caused by the bundle assortment in linear problem formulation, which will be illustrated in details in the following part

5 Algorithm details discussion

5.1 Relaxed Price Subadditivity Acceleration

Motivation. In Hanson’s original MILP, *price subadditivity* is enforced over the *entire* bundle set $\mathcal{B}_{\text{full}}$:

$$p_i \leq \sum_{j \in I} p_j, \quad \text{s.t.} \quad \bigcup_{j \in I} B(j) = B(i), \quad \forall i \in \{1, \dots, L\}. \quad (\text{A.10})$$

After we prune the search space to a much smaller feasible set $\mathcal{B}_{\text{fea}} \subset \mathcal{B}_{\text{full}}$, applying the same constraint to every missing bundle is unnecessary—and computationally expensive. Therefore, we *restrict* the subadditivity checks to $\mathcal{B}_{\text{fea}}$ only. Empirically, this change cuts hundreds of thousands of constraints while leaving the objective value virtually unchanged and reducing solution time significantly.

Two insertion rules. We explored two ways of re-introducing subadditivity inside $\mathcal{B}_{\text{fea}}$:

1. **Strict sub-bundle rule.** Add the inequality $p_i + p_j \geq p_k$ iff $B(i) \cup B(j) = B(k)$, $B(i) \subset B(k)$, $B(j) \subset B(k)$ and all three bundles $i, j, k \in \mathcal{B}_{\text{fea}}$.¹
2. **Minimal-cover rule.** For every $k \in \mathcal{B}_{\text{fea}}$ find *one* pair $(i, j) \in \mathcal{B}_{\text{fea}}^2$ such that $B(i) \cup B(j) = B(k)$ and $|B(i)| + |B(j)|$ is minimal; add only the single inequality $p_i + p_j \geq p_k$. This yields far fewer constraints.

Toy example. Suppose the pruned set contains four bundles

$$B(1) = \{1, 2\}, \quad B(2) = \{4, 5\}, \quad B(3) = \{1, 2, 5\}, \quad B(4) = \{5\}, \quad B(5) = \{1, 2, 4, 5\}, \quad B(6) = \{3, 4, 5\}.$$

- *Strict rule:* we add $p_1 + p_4 \geq p_3$ because $B(1) \cup B(4) = \{1, 2, 5\} = B(3)$.
- *Minimal-cover rule:* the same single constraint $p_1 + p_2 \geq p_5$ is added (it is already the least-sized cover), while no other pairs are used.

¹Inequalities are written in “ $\geq$ ” form for consistency with the Hanson notation $p_{b_1} + p_{b_2} \geq p_b$.

Empirical finding. Although the *strict* rule generates less constraints than the *minimal-cover* rule, hence strict rule obtain slightly higher revenue ratio and lower time ratio.

5.2 Problem Faced when Applying Bundle Assortment

A Algorithmic Strategies for Bundle Selection and Pricing

A.1 TPM: Thresholding + MILP

Core Idea:

- For each segment $j \in \{1, \dots, m\}$, apply a fixed threshold (0.5) to the predicted probability vector $P_j \in \mathbb{R}^n$ to determine its initial binary bundle assortment $B_j \in \{0, 1\}^n$, where $B_{ij} = 1$ if and only if $P_{ij} \geq 0.5$.
- The original full bundle space contains 2^n combinations of products. We construct a reduced feasible set $\mathcal{B}_{\text{pruned}} \subseteq \{0, 1\}^n$ of size m , where each bundle is formed by one segment's thresholded assortment.
- The pruned bundle pool $\mathcal{B}_{\text{pruned}}$ is shared across all segments in a global MILP formulation, where segments freely choose their preferred bundle from this reduced space.
- The final pricing problem is solved using Hanson's MILP model, which jointly optimizes segment-bundle assignments and corresponding prices.

Method: Mixed Integer Linear Programming (MILP) with bundle pruning. **Feasible Space:** m bundles, reused across segments.

Implementation Code:

```
# Predicted selection probabilities
prob = (model(data)[:n, :].sigmoid().numpy().T >= 0.5).astype(int)

# Solve via Hanson MILP with bundle pruning
ratio = revenue_ratio(n, segment_num, cs, Rs, Ns, opt_rev, prob)
```

A.2 MPM: Top- m Expansion + MILP

Core Idea:

- For each segment, sort products by descending predicted probability.
- Select top- m products and construct a sequence of prefix bundles:

$$[1], [1, 2], \dots, [1, 2, \dots, m].$$

- This generates $m \times m$ bundles across all segments, forming an expanded feasible space for MILP optimization.

Method: Mixed Integer Linear Programming (MILP) with bundle pruning. **Feasible Space:** $m \times m$ bundles, reused across segments.

Implementation Code:

```
fea_bundle = set()
for s in range(seg_num):
    top_idx = np.argsort(-prob[s, :])[:seg_num]
    for t in range(1, seg_num + 1):
        bundle = np.zeros(n, dtype=int)
        bundle[top_idx[:t]] = 1
        fea_bundle.add(bin2num(bundle))
ratio = revenue_ratio_B(n, seg_num, cs, Rs, Ns, opt_rev, fea_bundle)
```

Remarks:

- **Search space reduction:** thresholding reduces the original 2^n space to m bundles.
- **Segment-specific pruning, globally shared:** bundles are derived segment-wise but shared globally.
- **Scalability:** fixed feasible space size ensures scalability for large n .

A.3 PPM: Percentage Pruning + MILP

Core Idea:

- Set a percentage $\lambda\%$ and compute

$$k = \lceil n \times \lambda\% \rceil.$$

Select top- k products per segment, denote them as $\mathcal{K}_i$.

- For each segment, generate the power set:

$$\mathcal{P}(\mathcal{K}_i) = \{A \mid A \subseteq \mathcal{K}_i\}, \quad i = 1, \dots, m.$$

- The search space becomes:

$$\mathcal{B} = \bigcup_{i=1}^m \mathcal{P}(\mathcal{K}_i).$$

Method: MILP with bundle pruning. **Feasible Space:** $\leq m \times 2^k$, reused across segments.

Implementation Code:

```
def top_k_selection(output, k_percent=0.6):
    n, m = output.shape
    k = int(np.ceil(n * k_percent))
    selected_products = []
    for j in range(m):
        sorted_indices = np.argsort(output[:, j])[:, -1]
        top_k_indices = set(sorted_indices[:k].tolist())
        selected_products.append(top_k_indices)
    return selected_products
```

A.4 LPLS: Greedy Local Search + LP

Core Idea:

- Start from initial bundle predictions generated by thresholding $P_{ij} \geq 0.5$.
- For each segment, iteratively perform local perturbations:
 - **Addition move:** add the highest-probability unselected product.
 - **Removal move:** remove the lowest-probability selected product.
- Evaluate revenue improvement via an LP relaxation with fixed assignments.
- Accept changes only if revenue increases beyond $\varepsilon = 10^{-2}$.
- Stop when no improvement is possible; use the final bundles for LP pricing.

Pseudo-code:

Input: $P \in \mathbb{R}^{m \times n}$, $\tau = 0.5$

Output: Final revenue ratio and optimized bundles

1. $\text{best_pred} \leftarrow (P \geq \tau)$, $\text{best_rev} \leftarrow \text{LP}(\text{best_pred})$
2. Repeat until no improvement:
 - For each segment k :
 - Try adding product j^* with $\max P[k, j]$, $j \notin \text{bundle}$
 - Try removing product j^* with $\min P[k, j]$, $j \in \text{bundle}$
 - Accept change if LP revenue improves $> \varepsilon$
3. Return best_pred and solve MILP

Remarks:

- Greedy local search focuses on small perturbations per segment.
- LP provides fast revenue approximation.
- Faster than global MILP for small/medium n but may converge locally.

B MILP Formulations for Bundle Pricing

B.1 Hanson MILP Formulation

Variables: $(b \in \{1, \dots, B\}, k \in \{1, \dots, m\})$

- $\theta_{kb} \in \{0, 1\}$: $\mathbb{1}\{\text{Segment } k \text{ chooses bundle } b\}$
- p_b : Price of bundle b
- P_{kb} : Effective price paid by segment k for bundle b
- s_k : Surplus of segment k
- Z_{kb} : Profit from assigning bundle b to segment k

Objective:

$$\max_{\theta, p} \sum_{k=1}^m \sum_{b=1}^B N_k \cdot Z_{kb} \quad (\text{A.1})$$

Constraints:

$$\sum_{b=1}^B \theta_{kb} = 1 \quad \forall k \quad (\text{segment selects one bundle}) \quad (\text{A.2})$$

$$P_{kb} \leq p_b \quad \forall k, b \quad (\text{A.3})$$

$$P_{kb} \geq p_b - R_{\max}(1 - \theta_{kb}) \quad \forall k, b \quad (\text{A.4})$$

$$s_k = \sum_{b=1}^B (R_{kb} \theta_{kb} - P_{kb}) \quad \forall k \quad (\text{A.5})$$

$$Z_{kb} = P_{kb} - c_{kb} \theta_{kb} \quad \forall k, b \quad (\text{A.6})$$

$$p_b \leq p_{b_1} + p_{b_2}, \quad \text{if } A_b = A_{b_1} \cup A_{b_2} \quad (\text{subadditivity}) \quad (\text{A.7})$$

where $B = 2^n$ is the total number of bundles.

B.2 Bundle Size Pricing (BSP) MILP Formulation

Additional Variables:

- $\theta_{kb} \in \{0, 1\}$: $\mathbb{1}\{\text{Segment } k \text{ chooses bundle } b\}$
- p_s : Price for bundle size s , where $s \in \{0, 1, \dots, n\}$
- s_b : Size (cardinality) of bundle b
- $P_{kb} = p_{s_b} \theta_{kb}$

Objective:

$$\max_{\theta, p} \sum_{k=1}^m \sum_{b=1}^B N_k \cdot Z_{kb} \quad (\text{A.8})$$

Constraints:

$$\sum_{b=1}^B \theta_{kb} = 1 \quad \forall k \quad (\text{A.9})$$

$$P_{kb} = p_{s_b} \theta_{kb} \quad \forall k, b \quad (\text{A.10})$$

$$Z_{kb} = P_{kb} - c_{kb} \theta_{kb} \quad \forall k, b \quad (\text{A.11})$$

$$s_k = \sum_{b=1}^B (R_{kb} \theta_{kb} - P_{kb}) \quad \forall k \quad (\text{A.12})$$

$$p_{s_1+s_2} \leq p_{s_1} + p_{s_2} \quad \forall s_1, s_2 \text{ with } s_1 + s_2 \leq n \quad (\text{subadditivity}) \quad (\text{A.13})$$

$$p_{s+1} \geq p_s \quad \forall s \in \{0, \dots, n-1\} \quad (\text{A.14})$$

C Team Contribution Report

Experimental Design:

- **DLY:**
 - Independently developed Percentage Pruning Method.
 - Wrote Bundle Size Pricing code.
- **WCH:**
 - Independently developed Local Search Method.
- **DLY & WCH:**
 - Co-developed Threshold Pruning Method & TopM Pruning Method.
 - Jointly implemented the **constraint acceleration** technique.

Manuscript Writing:

- **DLY:**
 - Wrote the **MILP Formulations** section.
 - Wrote the **Proposed GCN Strategies** section.
 - Wrote the Percentage Pruning Method part in **Algorithm Details Discussion** section.
 - Conducted Numerical Experiments.
- **WCH:**
 - Wrote the **Background** section.
 - Wrote the **Strategy Comparison** section.
 - Wrote the **Experiment Results** section.
 - Wrote the **Algorithm Details Discussion** section.